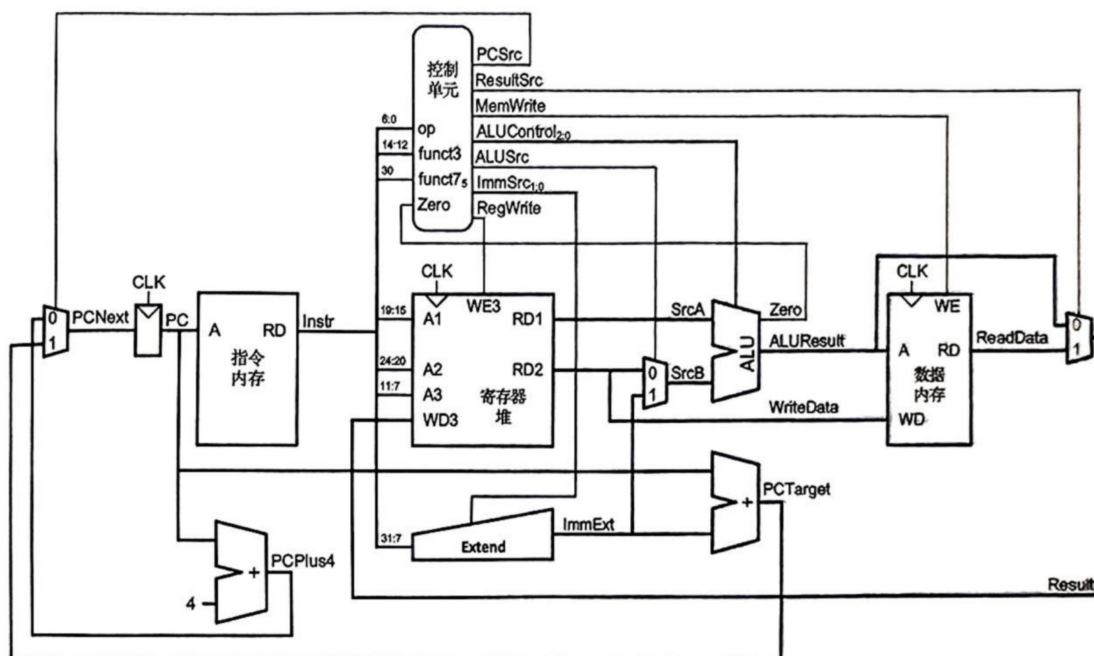


Homework_Chap7

在开始本次作业前，为便于描述，附两张图于此。



Review: Instruction Formats

7 bits	5 bits	5 bits	3 bits	5 bits	7 bits
funct7	rs2	rs1	funct3	rd	op
imm _{11:0}		rs1	funct3	rd	op
imm _{11:5}	rs2	rs1	funct3	imm _{4:0}	op
imm _{12,10:5}	rs2	rs1	funct3	imm _{4:1,11}	op
imm _{31:12}				rd	op
imm _{20,10:1,11,19:12}				rd	op
20 bits				5 bits	7 bits

R-Type
I-Type
S-Type
B-Type
U-Type
J-Type

(a) *RegWrite* 恒为1->寄存器堆写使能保持打开，会将 *Instr* 的7-11位以寄存器的形式译码并将 *Result* 的结果写入其中。会使S类型和B类型的指令出错，因其无需写寄存器，*Instr* 的7-11位参与组成立即数。若 *RegWrite* 为1，其将错误的写若干寄存器。

(b) *ALUOp₁* 恒为1->ALU的行为将由 *Instr* 的 *funct3* 字段和 *funct7* 字段决定。对于R类型和I-type ALU指令来说完全不受影响。对于B类型，讨论：首先beq指令肯定会被影响。正常情况下，ALU应当做减法操作；但根据 *funct3* (000) 解析出的ALU将会执行加法操作。换句话说，ALU将会以R类型指令的 *funct3* 和 *funct7* 决定所执行的运算。那么由于B类型的指令 *funct7* 字段用来组成立即数，所以除非beq指令为0100000xxxxxxxxx000xxxxx1100011时恰好ALU做减法，其余均会出错。对于除I-type ALU之外的I类型指令和S类型指令同B类型指令。

(c) *ALUOp₀* 恒为1->ALU将忠实的执行减法运算（只有B类型指令是进行两个寄存器相减）。B类型指令将不受影响。其余指令全错。

(d) *MemWrite* 恒为1->数据内存写使能保持打开，会将RD2的值忠实的写入 *ALU Result* 的地址。除了S类型外的其余指令均会意外的写入内存。似乎程序不会崩溃，除非写入的地址为只读字段等非法写入。但是数据内存将被意料之外的修改，可能导致后续访存指令读取错误数值。

(e) *ImmSrc₁* 恒为1->立即数均以B类型指令方式译码。除B类型指令和R类型指令（不涉及立即数）外，其余指令全错（立即数无法正确译码）。

(f) *ImmSrc₀* 恒为1->立即数均以S类型指令方式译码。除S类型指令和R类型指令外，其余指令全错。

(g) *ResultSrc₁* 恒为1->*Result* 均J类型指令的形式得到。除J类型指令外和B类型指令外，其余指令全错。B类型指令关注的点在 *PCNext* 的值，*Result* 是多少不是很重要。

(h) *ResultSrc₀* 恒为1->*Result* 均以访存的形式得到。R类型和I-type ALU类型的指令会出错，因为他们的 *Result* 是ALU运算的结果。

(i) *PCSrc* 恒为1->*PCNext* 的值均由 *PCTarget* 得到。除了J类型指令和B类型指令外全错。其余指令的 *PCNext* 是正常的 *PC* + 4得到。

(j) *ALUSrc* 恒为1->ALU的第二个操作数从 *Extend* 模块中来。所有R类型指令和B类型全错，因其ALU的两个操作数均来源于寄存器。

T7.3&7.4

先给出扩展后的指令真值表和数据通路。再针对不同指令进行解释。

指令	Op	RegWrite	ImmSrc	ALUSrc	MemWrite	ResultSrc	Bran
sw	0100011	0	01	1	1	xx	0
R-type	0110011	1	xx	0	0	00	0
beq	1100011	0	10	0	0	xx	1
I-type ALU	0010011	1	00	1	0	00	0
jal	1101111	1	11	x	0	10	0
bne	1110011	0	10	0	0	x	1
sra	0110011	1	xx	0	0	00	0
lbu	0000011	1	00	1	0	01	0
jalr	1100111	1	11	1	0	10	0
auipc	0010111	1	100?	1	0	00	0

- **bne**:当两个寄存器的值不同的时候，进行跳转。跳转的地址来自于 $Extend$ 。只需要将beq修改一下即可：检查 $func3$ 字段与ALU给出的 $Zero$ 信号，若前者为001且后者为0,则 $PCSrc$ 信号为1；否则为0。
- **sra**:一条简单的R-type指令，只需要修改ALU译码器的真值表，将其扩展为4位的控制信号并新支持一下sra指令就行。下面尝试给出修改后的ALU实现。

```

// ALU：组合逻辑，根据 ALUControl 执行运算
module ALU (
    input logic [31:0] A,
    input logic [31:0] B,
    input logic [2:0] ALUControl,
    output logic [31:0] Result,
    output logic Zero
);
    always_comb begin
        case (ALUControl)
            3'b000: Result = A + B; // add
            3'b001: Result = A - B; // sub
            3'b010: Result = A & B; // and
            3'b011: Result = A | B; // or
            3'b100: Result = A >>> B[4:0]; // sra (Shift Right Arithmetic)

            3'b101: Result = ($signed(A) < $signed(B)) ? 32'd1 : 32'd0; // slt
(signed)

            default: Result = 32'd0;
        endcase
    end

    assign Zero = (Result == 32'd0);

endmodule

```

- **lbu**:和lw类似，不过在读取内存的时候只读后8个bit。新增加一个控制信号 **RDMode**：真值表如下

func3	RDMode
000	10
100	00
101	01

当 **RDMode** 为10时，正常读word；为00时，读一个byte；为01时，读half word（2个byte）

- **jalr**:与jal指令不同之处仅在于，PC=rs1+imm。所以要修改 **PCSrc** 控制单元，将其扩展为2位：00表示PCPlus4，01表示PCTarget，10表示来自ALUResult。同时需要把ALUResult连接到最左侧的多路选择器。

- **auipc**:这条指令似乎要扩展 $immSrc$ 。具体来说，U类型指令立即数生成时应当取 $nstr$ 的12-31位，并将低12位置0。

T7.7

应当重新设计内存读单元。因为其在关键路径中会出现两次：更改后的单周期时间为：

$$100+100+120+100+30=450ps^{\circ}$$

T7.8

ALU延迟减少20ps，整体的单周期时间也减少20ps到730ps。执行一亿条指令所需时间为 $10^{11} \times 703 \div 10^{12} = 73秒$

T7.30

7.30		1	2	3	4	5	6	7	8	9
add	Fetch	decode \$s0 and \$s1	execute \$s0 and \$s1	Memory - no action	write \$t0					
sub		Fetch	decode \$t0 and \$s2	execute \$t0 - \$s2	Memory - no action	write \$t0				
lw			Fetch	decode \$t0	execute \$t0 + 60	Memory - \$t1 is read	write \$t1			
and				Fetch	decode \$t1 and \$t0	decode \$t1 and \$t0	execute \$t1 AND \$t0	Memory - no action	write \$t3	

ADD is stalled from cycle 5 to 6

寄存器操作有：读s2,s5；写s1

T7.31

7.31		1	2	3	4	5	6	7	8	9
add	Fetch	decode \$s0 and \$s1	execute \$s0 + \$s1	Memory - no action	write \$t0					
lw		Fetch	decode \$s2	execute 60 + \$s2	Memory - read [60+\$t2]	write \$t1				
sub			Fetch	decode \$t0 and \$s3	execute \$t0 - \$s3	Memory - no action	write \$t2			
and				Fetch	decode \$t1 and \$t0	execute \$t1 AND \$t0	Memory - no action	write \$t3		

寄存器操作有：读寄存器s0,写寄存器s1

T7.32

红色方框内为Stall的部分，是由于所读寄存器的值尚未写入造成的。

T7.33

同上题

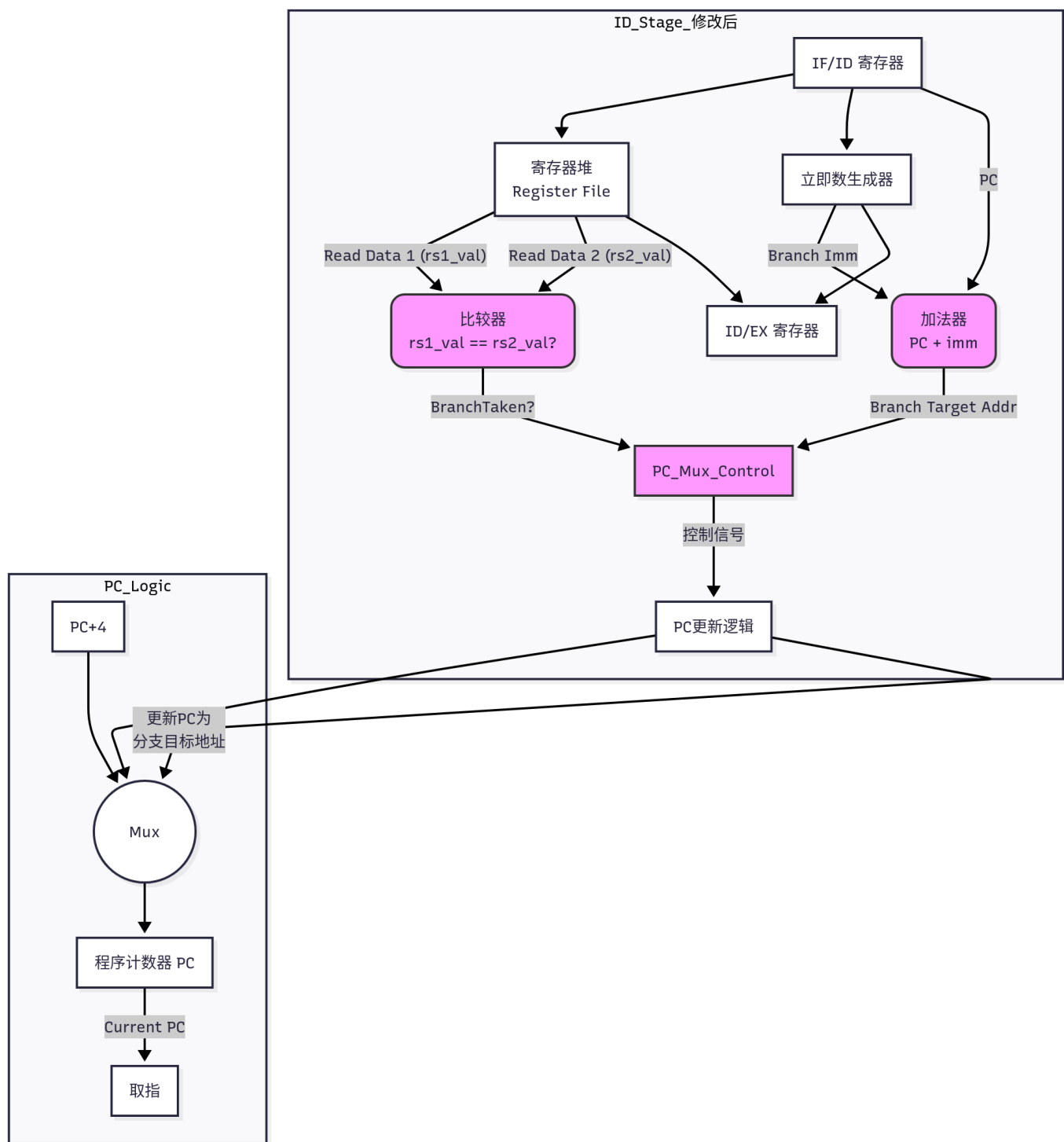
T7.34

时钟周期	1	2	3	4	5	6	7	8	9	10	11
I1: addi	IF	ID	EX	MEM	WB						
I2: lw s2		IF	ID	EX	MEM	WB					
I3: lw s5			IF	ID	stall	EX	MEM	WB			
I4: add s3				IF	ID	stall	EX	MEM	WB		
I5: or s4					IF	ID	EX	MEM	WB		
I6: and s2						IF	ID	EX	MEM	WB	

流水线图如上表所示。共需10个时钟周期，CPI=1.667

T7.38

正常情况下，分支指令（如beq之类）会在EX段计算目标地址并判断是否跳转。此时流水线中可能已经放入两条指令，分别为PC+4和PC+8（下一条指令进入ID阶段，下两条指令进入IF阶段）。此时若发生跳转，则两条指令都应当被刷新，代价为两个时钟周期。而当分支判断前移至ID阶段时，只有一条指令进入流水线，代价降低至一个周期。修改后的数据通路如图所示



但此时，前移分支逻辑引入了一种新的数据冒险。考虑以下指令序列：

```
lw    x5, 0(x10) // 在MEM段才能读出x5的值
beq   x5, x0, L1  // 在ID段就需要x5的值进行比较
```

- 当 **beq** 指令在ID段时，**lw** 指令在EX段。
- **lw** 指令的数据要到MEM段结束时才从内存中读出。
- 因此，**beq** 在ID段无法立即获得 **lw** 更新后的 **x5** 值。

所以，需要引入新的nop：当此时ID段的指令是跳转指令、且源寄存器是当前EX段的lw指令的目标寄存器，则要nop一个周期。等到lw的结果送到对应的寄存器中，此时才能做正确的比较。